

SOP Technical API Test

1. Scenario: The `/api/login` endpoint is used for user authentication and requires both email and password for successful login. However, incomplete login details should result in error responses.

Objective: Test both successful and failed login attempts and ensure the API handles missing fields appropriately.

Task:

- Write a test that:
 1. Sends a POST request with valid credentials (`{email: "eve.holt@reqres.in", password: "cityslicka" }`) and verifies that the response includes a 200 OK status and a valid token field.
 2. Sends a POST request with only the email (no password) and verifies the response status is 400 and the message is `{"error": "Missing password"}`.
 3. Sends a POST request without email and verifies the appropriate error is returned (`{"error": "Missing email or username"}`).
 4. Any validation related to headers that you may think?
 5. Do any other validations that you see fit.

API: <https://reqres.in/api/login>

2. Scenario: The `/api/users/2` endpoint returns details of a specific user.

Objective: Validate the data structure and specific content of the response.

Task:

- Write a test that:
 1. Sends a **GET** request to `/api/users/2` and verifies that the response includes the expected fields.
 2. Validate data format of the different attributes returned in the response
 3. Do any other validations that you may think of.

API: <https://reqres.in/api/users/2>

3. Scenario: The `/api/users` endpoint allows creating a user by sending a **POST** request with name and job fields.

Objective: Test the API's response to various invalid inputs when creating a user.

Task:

- Write a test that:
 1. Sends a POST request with valid name and job fields (e.g., name: "John", job: "developer") and validates that the user is successfully created (status 201).
 2. Do any other possible combinations that you may see fit, such as: missing fields (no name, no job), different possible headers, etc.
 3. Do any other validations that you may see fit.

API: <https://reqres.in/api/users>

4. Scenario: Validate pagination handling and the integrity of data across pages.

Objective: The `/api/users` endpoint returns paginated data, allowing clients to request different pages of users.

Task:

- Write a test that:
 1. Sends a GET request to `/api/users?page=2` and verifies the correct number of users is returned (default is 6 users per page).
 2. Validates that the page, **per_page**, total, and **total_pages** fields in the response contain accurate values.
 3. Fetches page 1 and page 2, and ensures that the users from the two pages are unique (i.e., no user appears on both pages).
 4. Attempts to access a non-existent page (e.g., `/api/users?page=999`) and verifies the API response handles this gracefully (should return an empty list or appropriate response).

API: <https://reqres.in/api/users>

5. Scenario: The `/api/users/{id}` endpoint is used to retrieve a user by id. When the user does not exist, the API should return a 404 Not Found.

Objective: Test the behavior of the API when trying to access a resource that does not exist.

Task:

- Write a test that:
 1. Sends a GET request to `/api/users/23` (a non-existent user) and verifies that the response status is 404 Not Found.
 2. Ensures that the response body is empty and no additional information is provided.
 3. Checks how the API handles malformed id values (e.g., `/api/users/abc`) and verifies whether an appropriate error or status code is returned.

API: <https://reqres.in/api/users/29>

6. Bonus Question 1: Chained requests for user list and specific user

Scenario: You want to first fetch a list of users and then retrieve detailed information about one of those users

Objective: Interact with the list of users and retrieve specific user details based on the list.

- Tasks:
 1. Think about the possible validations you could do here
 2. Have in mind clean code and reusability
 3. Validate that data from `api/users` is the same as `api/users/{id}`
 4. Do any other validations that you may think.
 5. Try to think any possible way of completing this task in a not correct way. This can be a couple of paragraphs (just text).

7. Bonus Question 2: Validate pagination and user consistency across pages

Scenario: You want to retrieve users from page 1 and page 2, ensuring that users are not duplicated across pages and that each user has valid field values.

Objective: Test pagination, ensuring user consistency and validating field structures across multiple pages of users.

- Task:
 1. Send a GET request to `/api/users?page=1` and validate that the response contains a list of users with fields `id`, `email`, `first_name`, `last_name`, and `avatar`.
 2. Send a GET request to `/api/users?page=2` and perform the same validation for the users on page 2.
 3. Ensure that no id from the user list on page 1 appears in the user list from page 2 (i.e., no duplicate users between pages).
 4. Check that each user's fields, such as email, `first_name`, and `last_name`, contain valid non-empty values.
 5. Verify that the total number of users per page matches the `per_page` field in the response (should be 6 users per page).

A HTML report must be generated and attached to the zip.

P.D: Please use some programming language or framework to do this and avoid using postman, jmeter or sending a txt file.